

AI/ML Engineer

Interview Questions & Answers

50 Curated Questions Covering ML Fundamentals, Deep Learning, NLP/LLMs, MLOps, Coding, and Behavioral Rounds

Contents

Machine Learning Fundamentals (Q1-Q10)
Deep Learning (Q11-Q20)
NLP / LLMs (Q21-Q28)
System Design / MLOps (Q29-Q38)
Coding / Practical (Q39-Q44)
Behavioral / Case-Based (Q45-Q50)

Machine Learning Fundamentals

Q1. Explain the bias-variance tradeoff. How do you diagnose which one is affecting your model?

Bias is error from overly simplistic assumptions (underfitting); variance is error from excessive sensitivity to training data (overfitting). High bias shows up as poor performance on both training and validation/test sets. High variance shows up as strong training performance but much weaker validation/test performance. You diagnose it by comparing training vs. validation error curves (learning curves) as data size or model complexity changes.

Q2. What's the difference between bagging and boosting?

Bagging (e.g., Random Forest) trains multiple models in parallel on bootstrapped samples and averages/votes their outputs to reduce variance. Boosting (e.g., XGBoost, AdaBoost) trains models sequentially, where each new model focuses on correcting the errors of the previous ones, primarily reducing bias. Bagging is more robust to overfitting; boosting often achieves higher accuracy but can overfit if not regularized.

Q3. How do you handle imbalanced datasets?

Techniques include resampling (oversampling minority class with SMOTE, undersampling majority class), using class weights in the loss function, choosing appropriate metrics (F1, PR-AUC, ROC-AUC instead of accuracy), ensemble methods designed for imbalance (e.g., Balanced Random Forest), and anomaly-detection framing when the minority class is extremely rare.

Q4. Explain precision, recall, and F1-score. When would you prioritize one over the other?

Precision = $TP/(TP+FP)$: of predicted positives, how many are correct. Recall = $TP/(TP+FN)$: of actual positives, how many were caught. F1 is the harmonic mean of both. Prioritize precision when false positives are costly (e.g., spam filtering blocking legitimate emails); prioritize recall when false negatives are costly (e.g., disease screening, fraud detection).

Q5. What is regularization, and how do L1 and L2 differ?

Regularization adds a penalty term to the loss function to discourage overly complex models and reduce overfitting. L1 (Lasso) adds the absolute value of coefficients, which can shrink some weights to exactly zero, producing sparse models and effectively performing feature selection. L2 (Ridge) adds the squared value of coefficients, shrinking weights smoothly toward zero without eliminating them, which handles multicollinearity well.

Q6. How does a Random Forest differ from Gradient Boosted Trees?

Random Forest builds many deep, independent trees in parallel on bootstrapped samples and averages predictions to reduce variance. Gradient Boosted Trees build shallow trees sequentially, each correcting the residual errors of the previous ones, reducing bias but requiring careful tuning of learning rate and tree depth to avoid overfitting. GBTs (like XGBoost/LightGBM) often outperform Random Forests on structured data but train more slowly and are more sensitive to hyperparameters.

Q7. What is cross-validation, and why is it important?

Cross-validation splits data into multiple train/validation folds (commonly k-fold) so the model is trained and evaluated on different subsets, giving a more reliable estimate of generalization performance than a single train/test split. It helps detect overfitting and supports robust hyperparameter tuning, especially with limited

data.

Q8. Explain the curse of dimensionality and how you'd address it.

As the number of features grows, data becomes increasingly sparse in the feature space, distances between points become less meaningful, and models require exponentially more data to generalize well. It's addressed through dimensionality reduction (PCA, t-SNE, UMAP), feature selection, regularization, and using models less sensitive to high dimensions (e.g., tree-based methods).

Q9. What's the difference between generative and discriminative models?

Discriminative models learn the decision boundary between classes directly, i.e., $P(y|x)$ — examples include logistic regression and standard neural network classifiers. Generative models learn the underlying data distribution, i.e., $P(x|y)$ and $P(y)$, and can generate new samples — examples include Naive Bayes, GANs, and diffusion models.

Q10. How do you choose the right evaluation metric for a business problem?

Start from the cost of different error types in the business context — false positives vs. false negatives don't usually cost the same. Translate that into metric choice (precision, recall, F1, AUC, RMSE, business KPIs like revenue lift), and align the metric with what stakeholders actually care about, not just what's easiest to optimize mathematically.

Deep Learning

Q11. Explain backpropagation in simple terms.

Backpropagation computes the gradient of the loss function with respect to each weight by applying the chain rule, propagating the error backward from the output layer to the input layer. These gradients are then used by an optimizer (like SGD or Adam) to update weights in the direction that reduces the loss.

Q12. What is vanishing/exploding gradient problem, and how do you mitigate it?

In deep networks, gradients can shrink toward zero (vanishing) or grow uncontrollably (exploding) as they propagate backward through many layers, making training unstable or ineffective. Mitigations include using ReLU-family activations instead of sigmoid/tanh, proper weight initialization (Xavier/He), batch normalization, gradient clipping, and architectures with skip connections like ResNets.

Q13. Compare CNNs, RNNs, and Transformers — when would you use each?

CNNs excel at spatially structured data like images by using convolutional filters to capture local patterns. RNNs (and LSTMs/GRUs) process sequential data by maintaining a hidden state over time, useful for shorter sequences where order matters. Transformers use self-attention to model relationships between all elements in a sequence in parallel, making them the standard for NLP and increasingly for vision and long-sequence tasks due to better scalability and parallelization.

Q14. What is the role of batch normalization?

Batch normalization normalizes layer inputs (mean 0, variance 1) within each mini-batch, which stabilizes and speeds up training, allows higher learning rates, and acts as a mild regularizer by reducing internal covariate shift.

Q15. Explain dropout and why it helps prevent overfitting.

Dropout randomly deactivates a fraction of neurons during each training iteration, forcing the network to not rely too heavily on any single neuron or co-adapted feature. This acts like training an ensemble of thinner sub-networks, which improves generalization at test time when all neurons are active.

Q16. What is attention, and why was it a breakthrough for sequence models?

Attention allows a model to dynamically weigh the importance of different parts of the input when producing each part of the output, rather than compressing the entire input into a single fixed-size context vector. This solved the bottleneck problem in RNN-based sequence-to-sequence models and enabled Transformers to capture long-range dependencies efficiently and in parallel.

Q17. How does a Transformer differ from an LSTM architecturally?

LSTMs process sequences step-by-step, maintaining a hidden state that carries information forward, which is inherently sequential and hard to parallelize. Transformers process the entire sequence at once using self-attention layers, allowing full parallelization during training and better modeling of long-range dependencies, at the cost of quadratic compute with sequence length.

Q18. What is transfer learning, and when would you use it vs. training from scratch?

Transfer learning reuses a model pretrained on a large dataset (e.g., ImageNet or a large text corpus) and fine-tunes it on a smaller, task-specific dataset. It's preferred when labeled data is limited or compute is constrained, since the pretrained model already captures general features. Training from scratch is reserved for cases with abundant data and a domain very different from anything the pretrained model has seen.

Q19. Explain the difference between fine-tuning and prompt engineering for LLMs.

Fine-tuning updates the model's weights using additional training data to adapt it to a specific task or domain, which requires compute and data but changes the model itself. Prompt engineering instead crafts the input (instructions, examples, formatting) to elicit better behavior from a frozen, pretrained model without any weight updates — faster and cheaper, but limited by what the base model already knows.

Q20. What are embeddings, and how are they used in ML systems?

Embeddings are dense vector representations of discrete objects (words, items, users, images) that capture semantic relationships, so similar objects end up close together in vector space. They're used for search/retrieval (via similarity search), recommendation systems, clustering, and as inputs to downstream models in NLP and other domains.

NLP / LLMs

Q21. How would you evaluate the quality of an LLM's output?

Evaluation combines automatic metrics (BLEU, ROUGE, perplexity for narrow tasks), model-based scoring (using another LLM as a judge), human evaluation for correctness/helpfulness/tone, and task-specific benchmarks. For production systems, it's important to also track hallucination rate, latency, and user feedback signals like thumbs up/down.

Q22. What is RAG (Retrieval-Augmented Generation), and when would you use it?

RAG retrieves relevant documents or passages from an external knowledge base (via vector search) and injects them into the LLM's prompt before generation, grounding the response in up-to-date or domain-specific information. It's used when you need factual accuracy, access to private/proprietary data, or frequently changing information without retraining the model.

Q23. Explain tokenization and its impact on model performance.

Tokenization splits text into subword units (via algorithms like BPE or WordPiece) that the model treats as its vocabulary. It affects how efficiently the model represents rare words, how many tokens a given input consumes (impacting cost and context window usage), and can introduce quirks in tasks like counting characters or arithmetic since tokens don't map cleanly to characters.

Q24. What are hallucinations in LLMs, and how do you reduce them?

Hallucinations occur when a model generates plausible-sounding but factually incorrect or fabricated content. They can be reduced through RAG (grounding responses in retrieved facts), fine-tuning on high-quality data, prompting the model to cite sources or say 'I don't know,' lowering generation temperature, and adding post-hoc fact-verification or guardrail layers.

Q25. How would you fine-tune an LLM for a domain-specific task?

Start by curating a high-quality, representative dataset for the target domain, then choose a fine-tuning approach — full fine-tuning for maximum adaptation (compute-heavy) or parameter-efficient methods like LoRA/QLoRA for cheaper, faster adaptation. Evaluate on held-out domain data, watch for catastrophic forgetting of general capabilities, and iterate with techniques like early stopping and learning rate scheduling.

Q26. What's the difference between zero-shot, few-shot, and fine-tuned approaches?

Zero-shot gives the model just an instruction with no examples, relying entirely on pretrained knowledge. Few-shot includes a handful of examples in the prompt to guide the model's behavior without updating weights. Fine-tuning actually updates model weights on task-specific data, generally yielding the best performance for narrow, high-volume tasks at the cost of more upfront effort.

Q27. How do you handle context length limitations in LLM applications?

Approaches include chunking documents and retrieving only the most relevant chunks (RAG), summarizing or compressing earlier conversation history, using sliding-window or hierarchical summarization for long documents, and choosing models with longer context windows when the use case truly requires it.

Q28. What is RLHF, and why is it used?

Reinforcement Learning from Human Feedback trains a reward model based on human preference rankings of model outputs, then uses that reward model to fine-tune the LLM via reinforcement learning (e.g., PPO). It's used to align model outputs with human values and preferences — making responses more helpful, honest, and harmless than what pretraining alone produces.

System Design / MLOps

Q29. How would you design a recommendation system for an e-commerce platform?

Typically a multi-stage pipeline: a candidate generation stage (collaborative filtering or embedding-based retrieval) narrows millions of items to hundreds, followed by a ranking stage (gradient boosted trees or a neural ranker) that scores candidates using rich features (user history, item metadata, context). Add business rules/filters (inventory, diversity), and continuously evaluate with offline metrics (NDCG, recall@k) and online A/B tests (CTR, conversion, revenue).

Q30. Walk me through your ML pipeline from data ingestion to deployment.

Data ingestion and validation, feature engineering (often via a feature store), train/validation/test splitting, model training and hyperparameter tuning, offline evaluation against baseline, packaging the model (containerization), staged rollout (shadow mode or canary deployment), monitoring in production for performance and drift, and a feedback loop for retraining.

Q31. How do you monitor a model in production for drift?

Track statistical distribution shifts in input features (data drift) using measures like population stability index or KL divergence, and monitor changes in the relationship between features and target (concept drift) via degrading prediction accuracy or proxy labels. Set up automated alerts when drift metrics cross thresholds, and maintain a retraining pipeline triggered by drift or scheduled cadence.

Q32. What's the difference between online and batch inference?

Batch inference processes large volumes of data at scheduled intervals and stores results for later use — efficient for non-time-sensitive predictions like periodic churn scoring. Online inference serves predictions in real time in response to individual requests, requiring low-latency infrastructure — used for use cases like fraud detection or live recommendations.

Q33. How would you design a real-time fraud detection system?

Combine a low-latency feature pipeline (streaming aggregations via something like Kafka + Flink) with a fast model (gradient boosted trees or a lightweight neural net) that scores transactions within milliseconds. Layer rule-based checks for known fraud patterns alongside the ML model, support human-in-the-loop review for borderline cases, and continuously retrain on newly confirmed fraud/non-fraud labels to keep up with evolving fraud tactics.

Q34. What is a feature store, and why is it useful?

A feature store is a centralized system for storing, managing, and serving features consistently across training and inference, solving the training-serving skew problem. It supports feature reuse across teams/models, versioning, and provides both offline (batch) and online (low-latency) access patterns.

Q35. How do you version datasets and models?

Datasets are versioned using tools like DVC or lakeFS, tying each dataset snapshot to a hash so experiments are reproducible. Models are versioned via a model registry (e.g., MLflow) that tracks metadata — training data version, hyperparameters, metrics, and lineage — allowing rollback and comparison between versions.

Q36. Describe your approach to A/B testing a new model against a production one.

Define the primary success metric upfront, calculate the required sample size for statistical power, randomly assign users to control (existing model) and treatment (new model) groups, run the test for a duration that accounts for weekly/seasonal patterns, and analyze results with appropriate significance testing while watching for guardrail metrics (latency, error rate) that shouldn't regress.

Q37. How would you scale model inference to handle millions of requests per day?

Techniques include model optimization (quantization, distillation, pruning), batching requests, using efficient serving frameworks (TensorRT, ONNX Runtime, Triton), horizontal scaling behind a load balancer, caching frequent predictions, and asynchronous or queue-based architectures for non-real-time workloads.

Q38. What's your approach to CI/CD for ML models?

Extend standard CI/CD with ML-specific stages: automated data validation and unit tests for pipeline code, automated model training and evaluation against a benchmark on each change, model registry integration for versioning, staged deployment (shadow/canary), and automated rollback triggers based on production monitoring metrics.

Coding / Practical

Q39. Implement k-means clustering from scratch.

Core logic: (1) randomly initialize k centroids, (2) assign each point to its nearest centroid using Euclidean distance, (3) recompute each centroid as the mean of points assigned to it, (4) repeat steps 2-3 until centroids stop changing significantly or a max iteration count is reached. Interviewers typically want to see correct distance computation, convergence handling, and awareness of centroid initialization sensitivity (mentioning k-means++ is a plus).

Q40. Write code to compute the confusion matrix without using a library.

Iterate through predicted and actual label pairs, incrementing counts into a matrix indexed by [actual][predicted]. For binary classification, explicitly track TP, TN, FP, FN counters by comparing each prediction/actual pair against the positive/negative class definitions, then assemble them into the 2x2 matrix.

Q41. Given a dataset with missing values, walk me through your preprocessing steps.

First, quantify missingness (percentage per column) and diagnose the mechanism (MCAR, MAR, MNAR) since that affects the right strategy. Options include dropping rows/columns with excessive missingness, simple imputation (mean/median/mode), model-based imputation (KNN or regression imputation), or adding a 'missingness indicator' feature when the fact that data is missing is itself informative. The choice should be validated by checking its effect on downstream model performance.

Q42. How would you optimize a slow-training model without more compute?

Options include reducing model/data precision (mixed-precision training), using a more efficient architecture or fewer parameters, subsampling or smarter batching of training data, early stopping, better data loading pipelines (avoiding I/O bottlenecks), gradient accumulation to simulate larger batches, and hyperparameter choices like a larger learning rate with warmup to converge faster.

Q43. Debug this: a model performs well in training but poorly in production — what could be wrong?

Likely causes: training-serving skew (features computed differently at inference time), data drift between training distribution and live traffic, data leakage during training that inflated training/validation metrics artificially, or bugs in the production feature pipeline. The debugging approach is to compare feature distributions and pipeline logic between training and serving environments side by side.

Q44. Write a function to split time-series data for training/validation without leakage.

The key principle is to always split chronologically — training data must come strictly before validation/test data in time, never using future information to predict the past. This typically means selecting a cutoff date/index and assigning everything before it to training and everything after to validation, optionally using multiple rolling or expanding windows (walk-forward validation) rather than a single random shuffle split.

Behavioral / Case-Based

Q45. Tell me about a project where your model didn't perform as expected. What did you do?

A strong answer walks through: what the initial expectation was, how the shortfall was diagnosed (error analysis, checking data quality, feature importance), what specific changes were made (new features, different model, more data, fixing a bug), and the measurable outcome afterward. Interviewers are looking for structured debugging and honesty about failure, not just a happy ending.

Q46. How do you communicate ML results to non-technical stakeholders?

Focus on business impact rather than technical metrics — translate accuracy/AUC into terms like revenue impact, cost savings, or risk reduction. Use visualizations over statistics tables, lead with the recommendation before the methodology, and proactively address limitations and confidence levels in plain language.

Q47. Describe a time you had to make a tradeoff between model accuracy and interpretability.

A good answer describes a specific context (e.g., a regulated industry like finance/healthcare, or a stakeholder who needed to trust and act on the output) where a simpler, more interpretable model (logistic regression, decision tree) was chosen over a higher-accuracy black-box model, and how that decision was justified and validated against business needs.

Q48. How do you stay updated with the fast-changing AI/ML landscape?

Common credible practices include following key research (arXiv, conference proceedings like NeurIPS/ICML), following practitioner communities and newsletters, reproducing papers or trying new tools hands-on, and participating in internal knowledge-sharing at work. The best answers show a habit of applying new knowledge, not just passive consumption.

Q49. Tell me about a time you disagreed with a teammate on a technical approach.

A strong answer shows the disagreement was resolved through data and experimentation rather than authority or opinion — e.g., running a quick benchmark or prototype to compare both approaches objectively, and being willing to adopt the other person's solution when the evidence supported it.

Q50. How would you approach a problem where you have very little labeled data?

Options include transfer learning from a pretrained model, data augmentation to synthetically expand the dataset, active learning to prioritize labeling the most informative examples, semi-supervised or self-supervised learning to leverage unlabeled data, and weak supervision (heuristics/rules) to generate noisy labels at scale.